

Recursive Descendant Parser

Syntax Analysis

Ahmad Yoosofan

Compiler course

University of Kashan

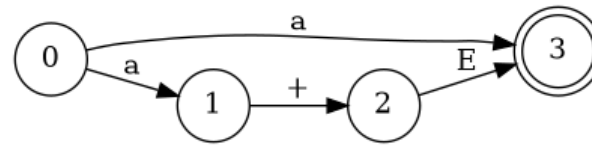
<https://yoosofan.github.io/course/compiler.html>

Special State Diagram for Addition

1. 2
2. 2+3
3. 86+54+876+432+32

Grammar

1. $E \rightarrow a$
 2. $E \rightarrow a + E$
- $E \rightarrow a + E \mid a$



Which edge must we choose from node 0?

Node 1

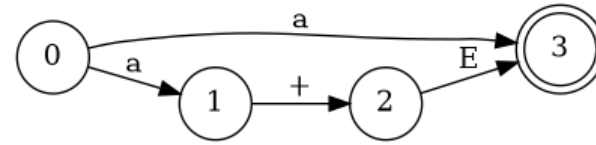
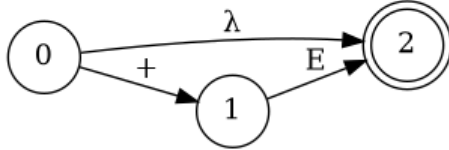
Node 3

Left Factoring

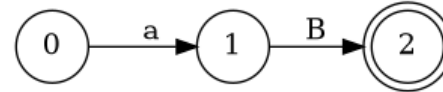
- $E \rightarrow a + E$
- $E \rightarrow a$

1. $E \rightarrow a B$
2. $B \rightarrow + E$
3. $B \rightarrow \lambda$

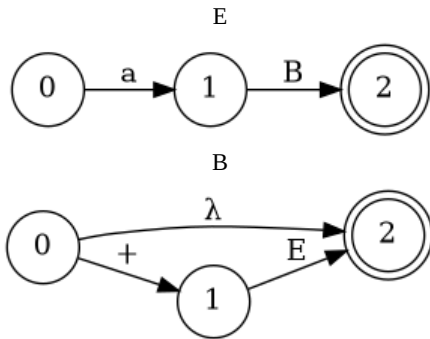
B



E



Parser Code for Add



```

1 def E(lex1):
2     t = lex1.getToken()
3     if t.type == 'a':
4         print('a B\t a: ', t.value, end=',')
5         print('\tB: ', lex1.s[lex1.end:])
6         return B(lex1)
7     print('Synatx Error: number expected')
8     print('Reminder: ', lex1.s[lex1.begin:])
9     return False
10
11 def B(lex1):
12     t = lex1.getToken()
13     if t.type == '+':
14         print('+E,\tE:', lex1.s[lex1.end:])
15         return E(lex1)
16     elif t.type == 'EOF': return True
17     print('Error a a:', s[0])
18     return False
19
20 def run(s):
21     lex1 = lexical_number_plus.lexical(s);
22     print(E(lex1))

```

```

python3 a.py 12.1 + 35.45 + 2
a B a: 12.1, B: +35.45+2
+E, E: 35.45+2
a B a: 35.45, B: +2
+E, E: 2
a B a: 2.0, B:
True

```

```

python3 a.py 12.1 + + 2
12.1 + + 2
a B a: 12.1, B: ++2
+E, E: +2
Synatx Error: number expected
Reminder: +2
False

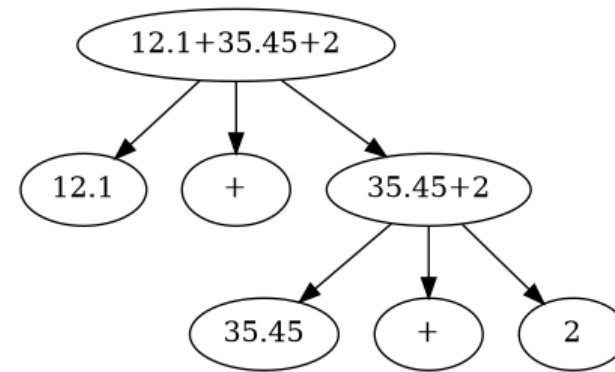
```

Parse Tree

```
1 def E(lex1):
2     t = lex1.getToken()
3     if t.type == 'a':
4         print('a B\t a: ', t.value, end=',')
5         print('\tB: ', lex1.s[lex1.end:])
6         return B(lex1)
7     print('Syntax Error: number expected')
8     print('Reminder: ', lex1.s[lex1.begin:])
9     return False
10
11 def B(lex1):
12     t = lex1.getToken()
13     if t.type == '+':
14         print('+E,\tE:', lex1.s[lex1.end:])
15         return E(lex1)
16     elif t.type == 'EOF': return True
17     print('Error a a:', s[0])
18     return False
19
20 def run(s):
21     lex1 = lexical_number_plus.lexical(s);
22     print(E(lex1))
```

```
python3 a.py 12.1 + 35.45 + 2
```

```
a B a: 12.1, B: +35.45+2
+E, E: 35.45+2
a B a: 35.45, B: +2
+E, E: 2
a B a: 2.0, B:
True
```



1. $E[12.1+35.45+2] \Rightarrow$
2. $a[12.1] + E[35.45+2] \Rightarrow$
3. $a[12.1] + a[35.45] + E[2] \Rightarrow$
4. $a[12.1] + a[35.45] + a[2] \Rightarrow$
5. Left Most Derivation

Doing Addition

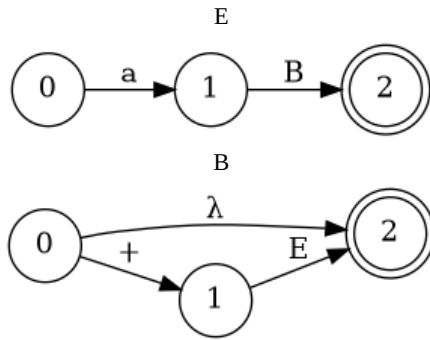
```
1 def E(lex1):
2     t = lex1.getToken()
3     if t.type == 'a':
4         print('a B\t a: ', t.value, end=',')
5         print('\tB: ', lex1.s[lex1.end:])
6         return B(lex1)
7     print('Synatx Error: number expected')
8     print('Reminder: ', lex1.s[lex1.begin:])
9     return False
10
11 def B(lex1):
12     t = lex1.getToken()
13     if t.type == '+':
14         print('+E,\tE:', lex1.s[lex1.end:])
15         return E(lex1)
16     elif t.type == 'EOF': return True
17     print('Error a a:', s[0])
18     return False
19
20 def run(s):
21     lex1 = lexical_number_plus.lexical(s);
22     print(E(lex1))
```

```
1 def E(lex1):
2     t = lex1.getToken()
3     if t.type == 'a':
4         print('a B\t a: ', t.value, end=',')
5         print('\tB: ', lex1.s[lex1.end:])
6         res = B(lex1)
7         return res[0], t.value + res[1]
8     print('Synatx Error: number expected')
9     print('Reminder: ', lex1.s[lex1.begin:])
10    return False, 0
11
12 def B(lex1):
13     t = lex1.getToken()
14     if t.type == '+':
15         print('+E,\tE:', lex1.s[lex1.end:])
16         return E(lex1)
17     elif t.type == 'EOF':
18         return True, 0
19     print('Error a a:', s[0])
20    return False, 0
```

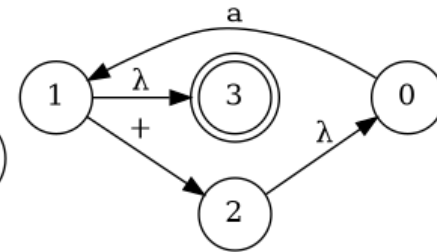
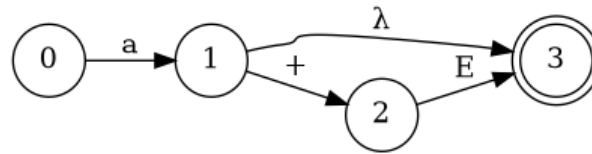
```
python3 a.py 12.1 + 35.45 + 2
a B a: 12.1, B: +35.45+2
+E, E: 35.45+2
a B a: 35.45, B: +2
+E, E: 2
a B a: 2.0, B:
(True, 49.550000000000004)
```

```
python3 a.py 2 +
a B a: 2.0, B: +
+E, E:
Synatx Error: number expected
Reminder:
(False, 2.0)
```

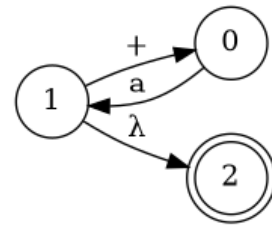
Simplify Diagram



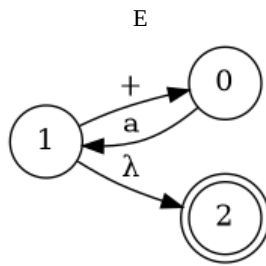
Combine them



E



Parser Code for add Nonrecursive

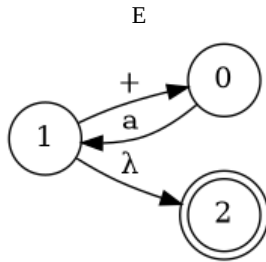


```
1 def E(lex1):
2     while True:
3         t = lex1.getToken()
4         if t.type == 'a':
5             print('a: ', t.value, end=',')
6             print('\t: ', lex1.s[lex1.end:])
7         else:
8             print('Synatx Error: number expected')
9             print('\t', lex1.s[lex1.begin:])
10            break
11        t = lex1.getToken()
12        if t.type != '+':
13            if t.type != 'EOF':
14                print('End of input expected')
15            break
16        if t.type != 'EOF': return False
17        return True
18
19 def run(s):
20     lex1 = lexical_number_plus.lexical(s);
21     print(E(lex1))
```

```
input: 16+34+0.30
output:
a: 16.0, : +34+0.30
a: 34.0, : +0.30
a: 0.3, :
True
```

```
input: 23+ + 34
output:
a: 23.0, : ++34
Synatx Error: number expected
+34
False
```

Calculator Code for add Nonrecursive

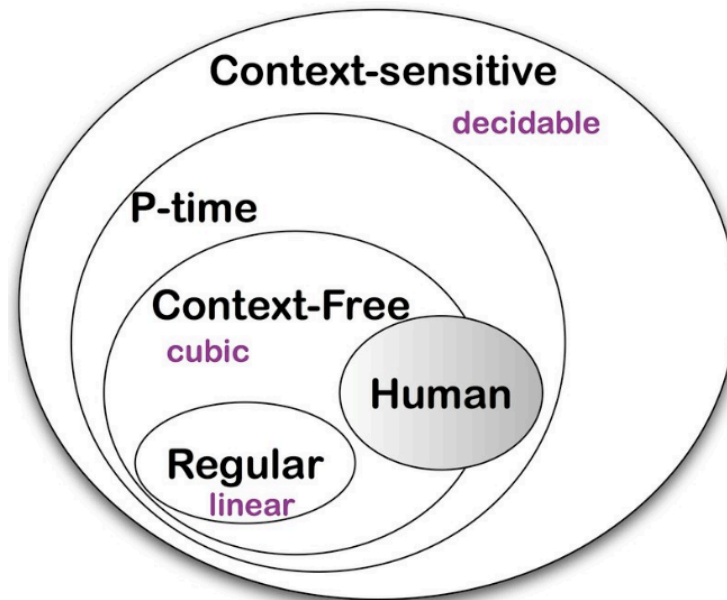


```
1 def E(lex1):
2     x = 0.0; state = 0; validity = True;
3     while True:
4         if state == 0:
5             t = lex1.getToken()
6             if t.type == 'a':
7                 print('a: ', t.value, end=',')
8                 print('\t: ', lex1.s[lex1.begin:]); validity = False; break
9                 state = 1; x += t.value;
10            else:
11                print('Synatx Error: number expected')
12                print('\t', lex1.s[lex1.begin:]); validity = False; break
13        elif state == 1:
14            t = lex1.getToken()
15            if t.type == '+': state = 0
16            elif t.type == 'EOF': state = 2
17            else: print('End of input expected'); validity = False; break;
18        elif state == 2: validity = True; break
19    return validity, x
```

```
input: 16+34+0.30
output:
a: 16.0, : +34+0.30
a: 34.0, : +0.30
a: 0.3, :
(True, 50.3)
```

```
input: 23+ + 34
output:
a: 23.0, : ++34
Synatx Error: number expected
+34
(False, 23.0)
```

Classes of Languages



https://www.researchgate.net/figure/Human-languages-and-the-classes-of-the-Chomsky-hierarchy-with-parsing-complexity_fig1_257299091

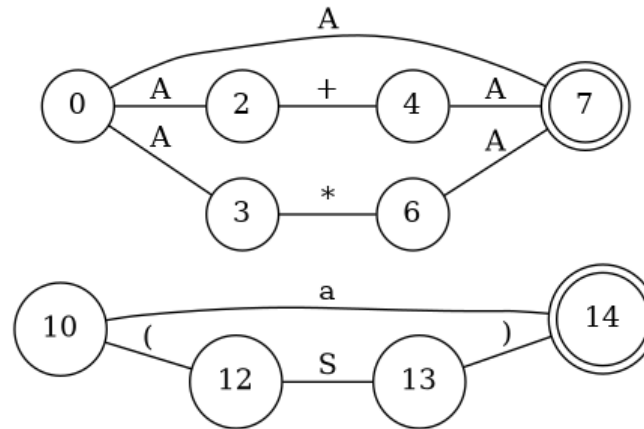
Parse for + * () no priority

Consider the following sentences of a language:

Write down the Grammar

1. 3
2. 9 + 3
3. 9 * 3
4. (9 * 3)
5. (9 + 3)
6. (9 * 4) + 5
7. 9 * (4 + 5)
8. (9 * (4 + 5)) * 3
9. 9 * ((4 + 5) * 3)
10. 9 * ((4 + 5) + 3)
11. (9 + (4 + 5)) * 3

1. $S \rightarrow A + A$
2. $S \rightarrow A * A$
3. $S \rightarrow A$
4. $A \rightarrow (S)$
5. $A \rightarrow a$

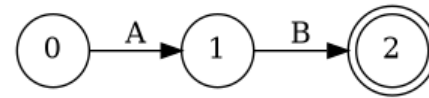


Left Factoring

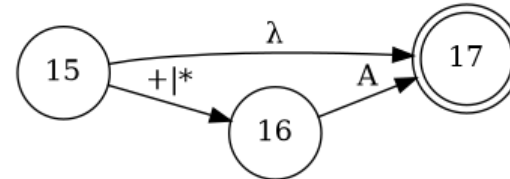
1. $S \rightarrow A + A$
2. $S \rightarrow A * A$
3. $S \rightarrow A$
4. $A \rightarrow (S)$
5. $A \rightarrow a$

1. $S \rightarrow A B$
2. $B \rightarrow +A \mid *A \mid \lambda$
3. $A \rightarrow (S) \mid a$

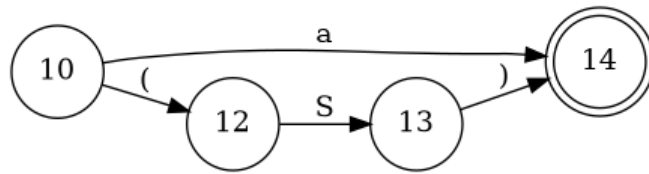
- S



- B

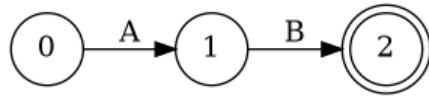


• A

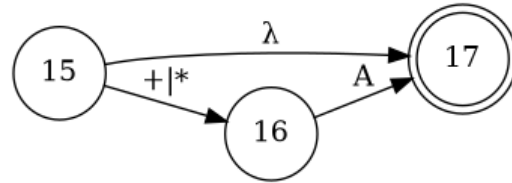


Simplify Diagram

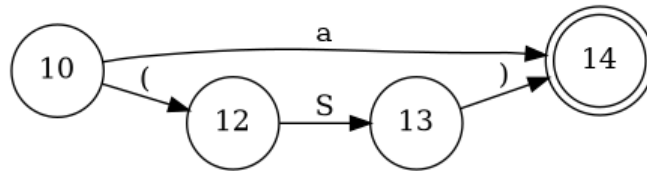
- S



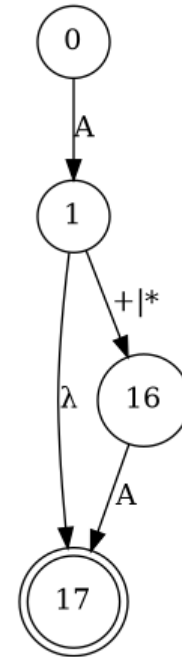
- B



- A

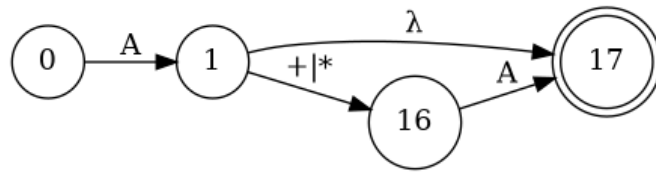


- Combine S and B

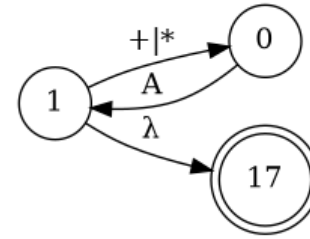
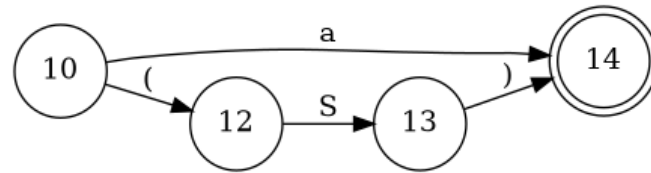


Remove Some Recursion

- S

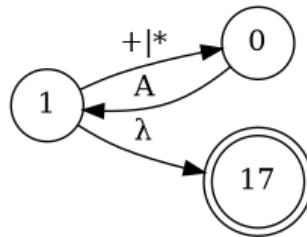


- A

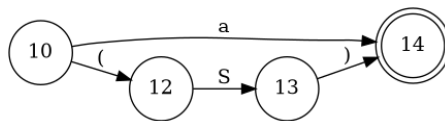


Parser Code

• S



• A



```

1 def S(lex1):
2     while True:
3         if A(lex1) == False: return False
4         if lex1.ct.type != '+' and lex1.ct.type != '*':
5             break
6         print(lex1.ct.type, ': ', lex1.s[lex1.end:])
7         lex1.getToken()
8         if lex1.ct.type == 'EOF' or lex1.ct.type == ')':
9             return True
10        return False
11 def A(lex1):
12     if lex1.ct.type == 'a':
13         print('a: ', lex1.ct.value, end=',')
14         print('\t: ', lex1.s[lex1.end:])
15         lex1.getToken()
16     elif lex1.ct.type == '(':
17         print('\t: ', lex1.s[lex1.end:])
18         lex1.getToken()
19         if S(lex1) == False: return False
20         if lex1.ct.type != ')' :
21             print('Syntax Error: ) expected')
22             print(lex1.ct.type, '\t', lex1.s[lex1.begin:])
23             return False
24         else: print(''); lex1.getToken()
25     else:
26         print(' a or ( expected:: ', lex1.ct.type)
27         return False;
28     return True
29 def run(s):
30     lex1 = lexical_number_plus_mul_par.lexical(s);
31     lex1.getToken()
32     if S(lex1) == True:
  
```

```

1 import sys, lexical_number_plus_mul_par
2 def S(lex1):
3     while True:
4         if A(lex1) == False: return False
5         if lex1.ct.type != '+' and lex1.ct.type != '*':
6             break
7         print(lex1.ct.type, ': ', lex1.s[lex1.end:])
8         lex1.getToken()
9         if lex1.ct.type == 'EOF' or lex1.ct.type == ')':
10            return True
11        return False
12 def A(lex1):
13     if lex1.ct.type == 'a':
14         print('a: ', lex1.ct.value, end=',')
15         print('\t: ', lex1.s[lex1.end:])
16         lex1.getToken()
17     elif lex1.ct.type == '(':
18         print('(\t: ', lex1.s[lex1.end:])
19         lex1.getToken()
20         if S(lex1) == False: return False
21         if lex1.ct.type != ')':
22             print('Syntax Error: ) expected')

```

```

21         print(lex1.ct.type, '\t', lex1.s[lex1.begin:
22             return False
23         else: print(')'); lex1.getToken()
24     else:
25         print(' a or ( expected: ', lex1.ct.type)
26         return False;
27     return True
28 def run(s):
29     lex1 = lexical_number_plus_mul_par.lexical(s);
30     lex1.getToken()
31     if S(lex1) == True:
32         if lex1.ct.type == 'EOF':
33             print(True)
34         else:
35             print('Syntax Error: extra characters')
36             print(lex1.s[lex1.begin:])
37     else:
38         print('Syntax Error at')
39         print(lex1.s[lex1.begin:])
40 if __name__ == "__main__":
41     if len(sys.argv) < 2:
42         print("python3 a.py '12.1 + (35.45 * 2)'" )
43         raise SystemExit(f"Usage: {argvs[0]} input exp
44     s = ''.join(sys.argv[1:])
45     run(s)

```

Lexical

```
2 class tokenCls:
3     def __init__(self, type1='EOF',
4         v1='', l1=0, b1=0, e1=0):
5         self.type = type1
6         self.begin = b1
7         self.end = e1
8         self.value = v1
9         self.lineno = l1
10    def __str__(self):
11        return '[type: ' + str(self.type) + \
12            '\tvalue: ' + str(self.value) + \
13            '\tline: ' + str(self.lineno) + \
14            '\tbegin: ' + str(self.begin) + \
15            '\tend: ' + str(self.end) + ']'
16 class lexical:
17     def __init__(self, s):
18         self.s = s
19         self.begin = 0
20         self.end = 0
21         self.lineno = 0
22         self.length = len(self.s)
23         self.ct = tokenCls(b1=self.begin,
24             e1=self.end)
25     def getToken(self):
26         self.ct = tokenCls(b1=self.begin,
27             e1=self.end)
28         self.begin = self.end
29         while self.end < self.length:
30             a = re.match('[ \t]+', self.s[self.end:])
31             if a is not None:
32                 self.end = self.begin = self.end + a.end()
33             if self.end >= self.length:
```

```
33         break
34
35         a = re.match('\\d+(\\.\\d+)?',
36             self.s[self.end:])
37         if a is None:
38             self.end += 1
39         operators1 = ['+', '-', '*', '/', '(', ')']
40         if self.s[self.begin:self.end] in operators:
41             self.ct.type = self.s[self.begin:self.end]
42             break
43         else:
44             print('Lexical error: ', self.lineno, end=
45                 '\n', self.begin, ': ', self.end, end=
46                     '\n', self.s[self.begin:self.end],
47                         self.begin += 1)
48         else:
49             self.ct.type = 'a'
50             self.ct.value = float(a.group())
51             self.end += a.end()
52             break
53         self.ct.begin = self.begin
54         self.ct.end = self.end
55         self.ct.lineno = self.lineno
56         return self.ct
57 if __name__ == '__main__':
58     #s = input('Enter ') # 12+35+2
59     s = '12+35+2 '
60     s = ' 12 + 35 + 2 '
61     s = ' 12 + 35 --- 2 '
62     # ~ s = '1.0.' s = '1.0=23.2+4'
63     s = '12+35+2 '
64     lex1 = lexical(s);
```

Error Recovery (panic mode)

A Simple Calculator (,), +, *

1. 34
2. 34 + 2
3. 34 + 45 + 98
4. 34 + 45+98 * 4 * 554
5. (34 * 2) + 3
6. 43 * (54+3)
7. 2 + (34)
8. (34+3) * 2
9. 34+3 * 2 * ((4))

Grammar 1

1. $S \rightarrow S + S$
2. $S \rightarrow S * S$
3. $S \rightarrow a$
4. $S \rightarrow (S)$

Incorrect Grammar (Why?)

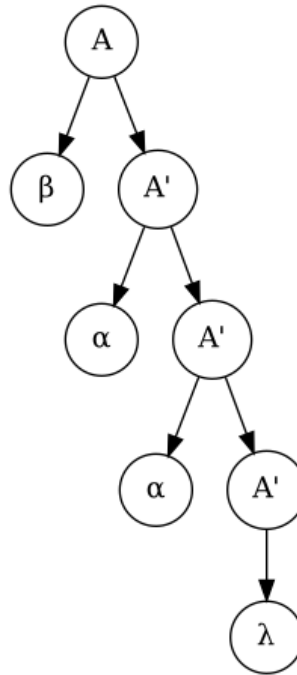
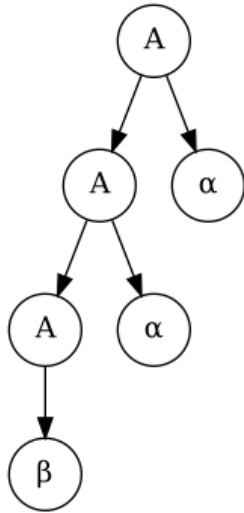
Grammar 2

1. $E \rightarrow T + E \mid T$
2. $T \rightarrow F * T \mid F$
3. $F \rightarrow (E) \mid a$

Grammar 3

- $E \rightarrow E + T \mid T$
- $T \rightarrow T * F \mid F$
- $F \rightarrow (E) \mid a$

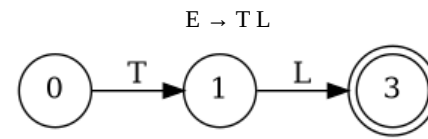
Left Recursion Elimination, $A \rightarrow A \alpha \mid \beta, \beta \alpha \alpha$

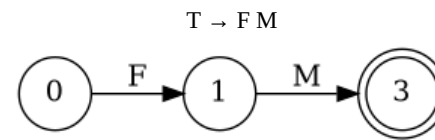
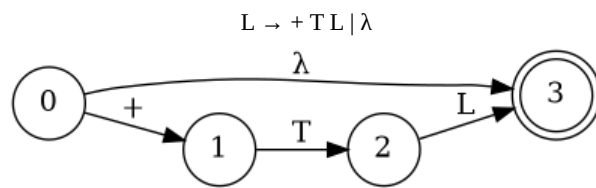


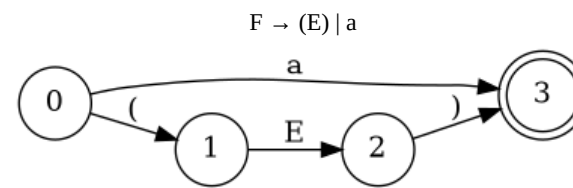
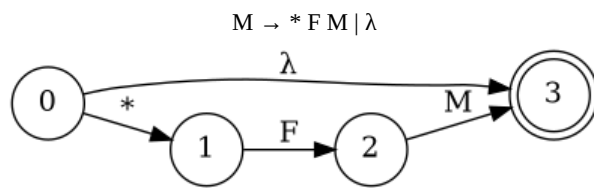
- $A \Rightarrow A \alpha \Rightarrow A \alpha \alpha \Rightarrow \beta \alpha \alpha$
- $A \Rightarrow \beta A' \Rightarrow \beta \alpha A' \Rightarrow \beta \alpha \alpha A' \Rightarrow \beta \alpha \alpha$
- $A \rightarrow A \alpha \mid \beta$
 1. $A \rightarrow \beta A'$
 2. $A' \rightarrow \alpha A' \mid \lambda$
- $E \rightarrow E + T \mid T$
 1. $E \rightarrow T L$
 2. $L \rightarrow + T L \mid \lambda$
- $T \rightarrow T * F \mid F$
 1. $T \rightarrow F M$
 2. $M \rightarrow * F M \mid \lambda$
- $F \rightarrow (E) \mid a$

State Diagram(II)

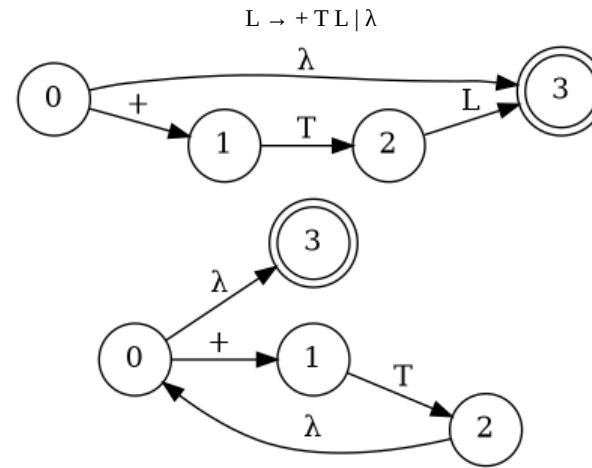
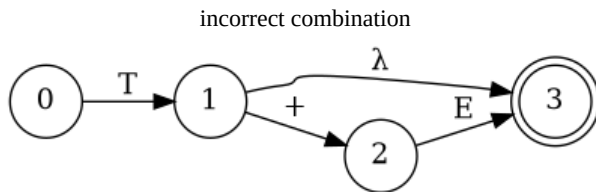
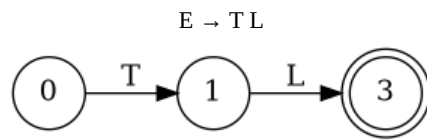
1. $E \rightarrow T L$
2. $L \rightarrow + T L \mid \lambda$
3. $T \rightarrow F M$
4. $M \rightarrow * F M \mid \lambda$
5. $F \rightarrow (E) \mid a$

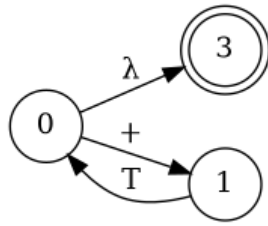




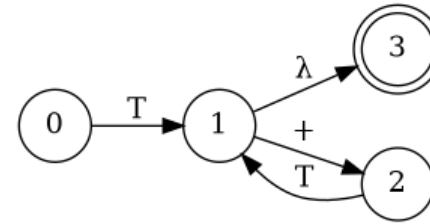


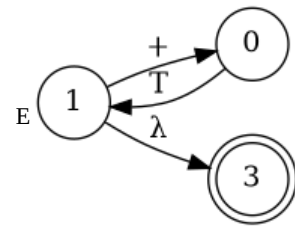
Simplify Diagram(II) - Combine E, L





Combine $E \rightarrow T L$ and simplified chart of $L \rightarrow + T L \mid \lambda$

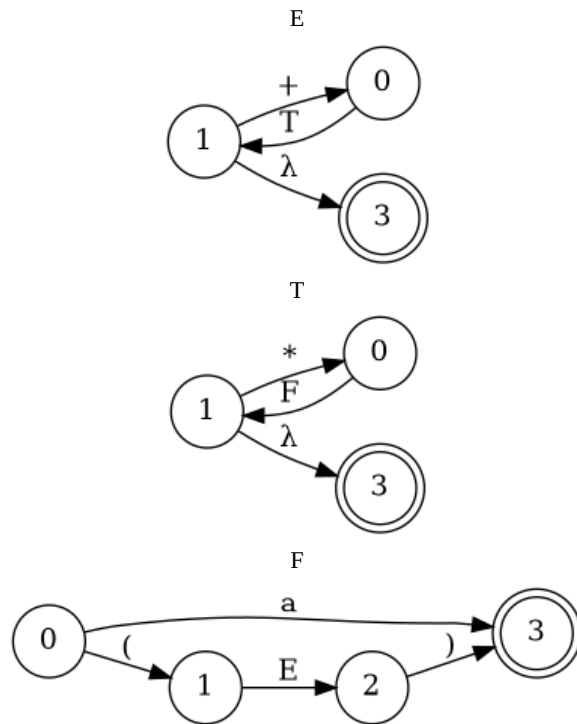




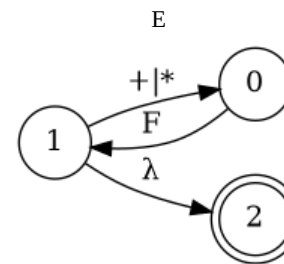
- Do the same for the following grammar rules

1. $T \rightarrow F M$
2. $M \rightarrow * F M \mid \lambda$

Diagrams of the Second Calculator

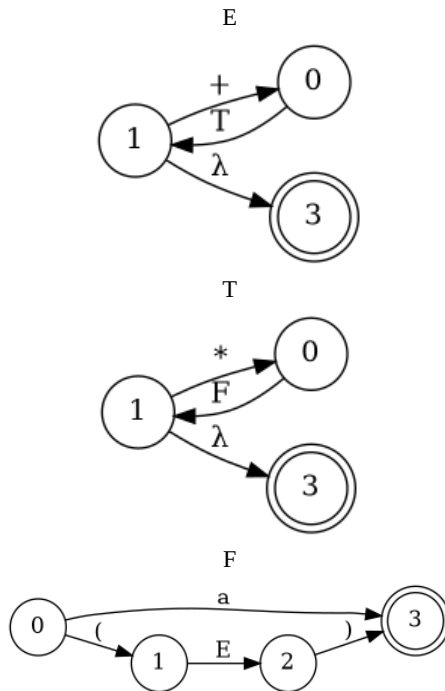


Can E and T be combined?



1. $4 + 3 * 2$
2. $(4 + 3) * 2$
3. $4 + (3 * 2)$
4. $3 * 2 + 4$
5. $3 * (2 + 4)$
6. $(3 * 2) + 4$

Parser Code for Add and Multiply



```

1 def S(lex1):
2     lex1.getToken()
3     if E(lex1) == True:
4         if lex1.ct.type == 'EOF': print(True)
5     else:
6         print('Syntax Error: extra characters at END: ')
7         print(lex1.s[lex1.begin:])
8     else: print('Syntax Error at', lex1.s[lex1.begin:])
9 def E(lex1):
10    while True:
11        if T(lex1) == False: return False
12        if lex1.ct.type != '+': break
13        lex1.getToken()
14    return True
15 def T(lex1):
16    while True:
17        if F(lex1) == False: return False
18        if lex1.ct.type != '*': break
19        lex1.getToken()
20    return True;
21 def F(lex1):
22     if lex1.ct.type == 'a':
23         lex1.getToken()
24     elif lex1.ct.type == '(':
25         lex1.getToken()
26         if E(lex1) == False: return False
27         if lex1.ct.type != ')': return False
28         lex1.getToken()
29     else: return False
30    return True

```

Another Language for simple expression $+ - * / ()$

1. $2 + 3 * 4 - 5$
2. $4 - 5 * 2 / (4 - 2) + 1$
3. $((2 * (3 - 1)) / (5 - 3)) * (7 - 8)$

Grammar ?

1. $E \rightarrow T + E \mid T - E \mid T$
2. $T \rightarrow F * T \mid F / T \mid F$
3. $F \rightarrow (E) \mid a$

Is there any problem?

1. $4 - 3 - 2$
2. $16 / 4 / 2$

It was a incorrect grammar

1. $E \rightarrow E + T \mid E - T \mid T$
2. $T \rightarrow T * F \mid T / F \mid F$
3. $F \rightarrow (E) \mid a$

Left Most Derivation

1. $E \Rightarrow E + T \Rightarrow$
2. $T + T \Rightarrow$
3. $F + T \Rightarrow$
4. $16 + T \Rightarrow 16 + T * F \Rightarrow$
5. $16 + F * F \Rightarrow$
6. $16 + 3 * F \Rightarrow$
7. $16 + 3 * 4$

General Form of Direct Left Recursion Elimination

$$A \rightarrow A \alpha_1 \mid A \alpha_2 \mid \dots \mid A \alpha_m \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

- $A \rightarrow A \alpha \mid \beta$
 1. $A \rightarrow \beta A'$
 2. $A' \rightarrow \alpha A' \mid \lambda$

Convert to

- $A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_n A'$
- $A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_m A' \mid \lambda$

Calculator Grammar

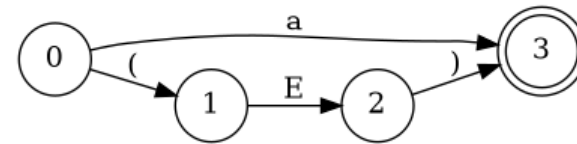
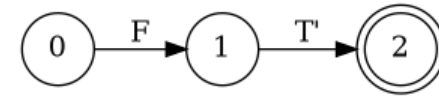
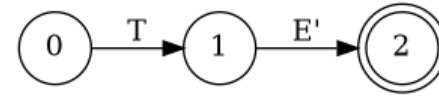
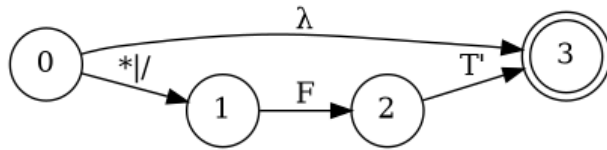
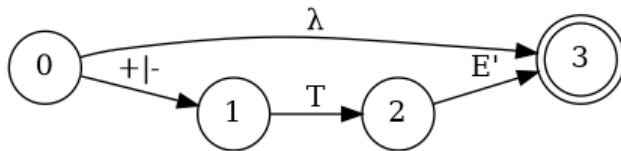
- $E \rightarrow E + T \mid E - T \mid T$
- $T \rightarrow T * F \mid T / F \mid F$
- $F \rightarrow a \mid (E)$

Convert to

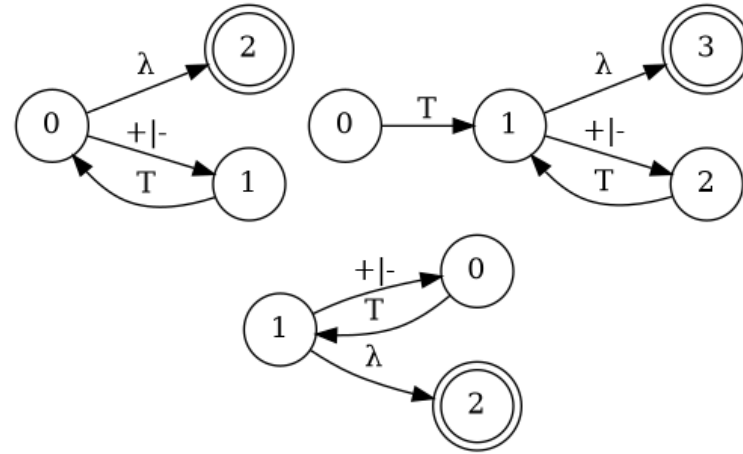
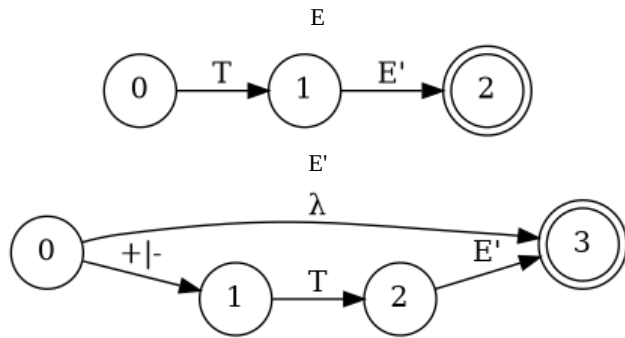
- $E \rightarrow T E'$
- $E' \rightarrow + T E' \mid - T E' \mid \lambda$
- $T \rightarrow F T'$
- $T' \rightarrow * F T' \mid / F T' \mid \lambda$
- $F \rightarrow a \mid (E)$

Calculator Grammar

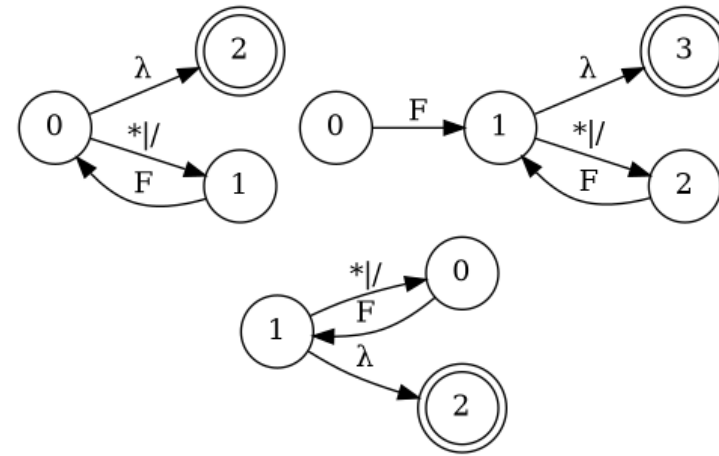
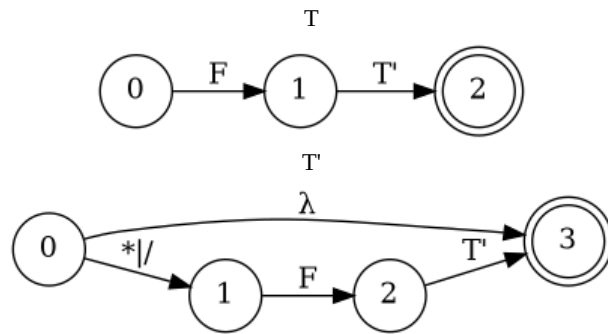
- $E \rightarrow TE'$
- $E' \rightarrow +TE' \mid -TE' \mid \lambda$
- $T \rightarrow FT'$
- $T' \rightarrow *FT' \mid /FT' \mid \lambda$
- $F \rightarrow a \mid (E)$



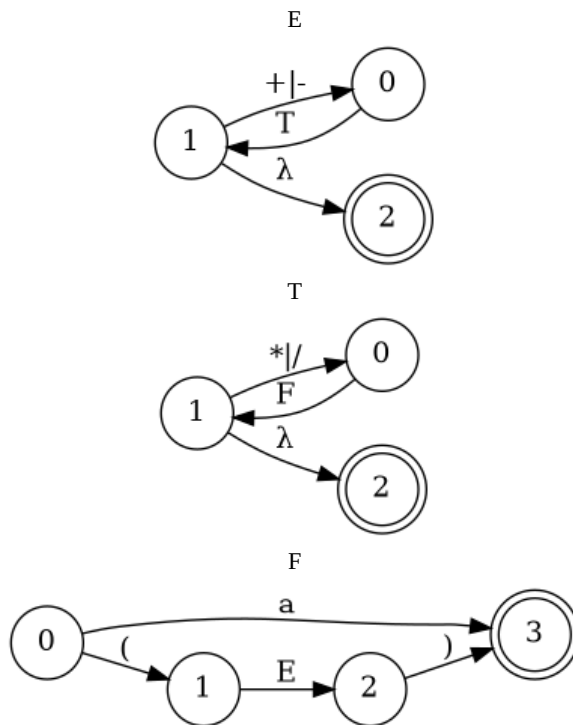
Simplify Diagram(V) - Combine E and E'



Simplify Diagram(VI) - Combining T and T'



Parser Code for Last Calculator



```

1 def S(lex1):
2     lex1.getToken()
3     if E(lex1) == True:
4         if lex1.ct.type == 'EOF': print(True)
5     else:
6         print('Syntax Error: extra characters at END: ')
7         print(lex1.s[lex1.begin:])
8     else: print('Syntax Error at', lex1.s[lex1.begin:])
9 def E(lex1):
10    while True:
11        if T(lex1) == False: return False
12        if lex1.ct.type not in ['+', '-']: break
13        lex1.getToken()
14    return True
15 def T(lex1):
16    while True:
17        if F(lex1) == False: return False
18        if lex1.ct.type not in ['*', '/']: break
19        lex1.getToken()
20    return True;
21 def F(lex1):
22    if lex1.ct.type == 'a':
23        lex1.getToken()
24    elif lex1.ct.type == '(':
25        lex1.getToken()
26        if E(lex1) == False: return False
27        if lex1.ct.type != ')': return False
28        lex1.getToken()
29    else: return False
30    return True

```

Output Parser Tree

```
9 def E(lex1):
10     while True:
11         print('E: ', lex1.s[lex1.begin:])
12         if T(lex1) == False: return False
13         if lex1.ct.type not in ['+', '-']:
14             break
15         print('+-: ', lex1.s[lex1.begin:])
16         lex1.getToken()
17     return True
18 def T(lex1):
19     while True:
20         print('T: ', lex1.s[lex1.begin:])
21         if F(lex1) == False: return False
22         if lex1.ct.type not in ['*', '/']:
23             break
24         print('*/: ', lex1.s[lex1.begin:])
25         lex1.getToken()
26     return True;
27 def F(lex1):
28     print('F: ', lex1.s[lex1.begin:])
29     if lex1.ct.type == 'a':
30         print('Number: ', lex1.ct.value)
31         lex1.getToken()
32     elif lex1.ct.type == '(':
33         lex1.getToken()
34         if E(lex1) == False: return False
35         if lex1.ct.type != ')':
36             return False
37         lex1.getToken()
38     else: return False
39     return True
```

```
1  rd$python3 e_t_f_plus_minus_mul_divide_parser_tree.py
2  Enter an expression like 1+4*(3-1)
3  > 1+4*(3-1)
4  E:  1+4*(3-1)
5  T:  1+4*(3-1)
6  F:  1+4*(3-1)
7  Number:  1.0
8  +-:  +4*(3-1)
9  E:  4*(3-1)
10 T:  4*(3-1)
11 F:  4*(3-1)
12 Number:  4.0
13 */:  *(3-1)
14 T:  (3-1)
15 F:  (3-1)
16 E:  3-1)
17 T:  3-1)
18 F:  3-1)
19 Number:  3.0
20 +-:  -1)
21 E:  1)
22 T:  1)
23 F:  1)
24 Number:  1.0
25 True
26 Enter an expression like 1+4*(3-1)
27 >
```

Representations of Parse Tree

```

> 1+4*(3-1)
E: 1+4*(3-1)
T: 1+4*(3-1)
F: 1+4*(3-1)
Number: 1.0
+-: +4*(3-1)
E: 4*(3-1)
T: 4*(3-1)
F: 4*(3-1)
Number: 4.0
*/: *(3-1)
T: (3-1)
F: (3-1)
E: 3-1)
T: 3-1)
F: 3-1)
Number: 3.0
+-: -1)
E: 1)
T: 1)
F: 1)
Number: 1.0
True
    
```

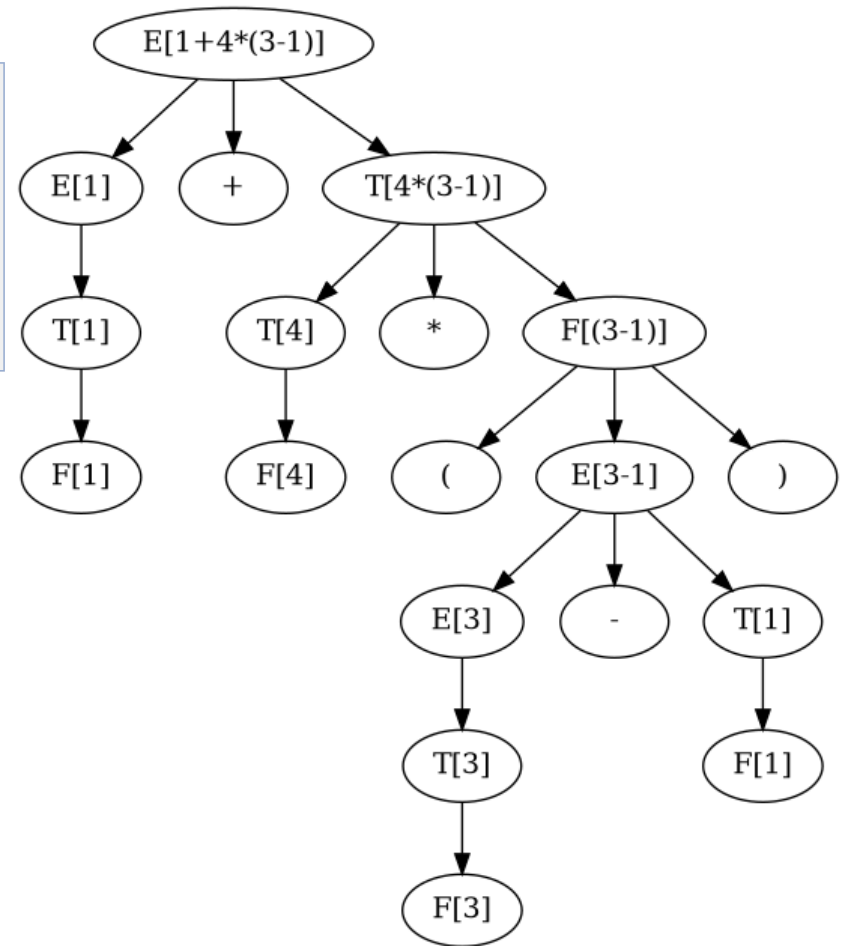
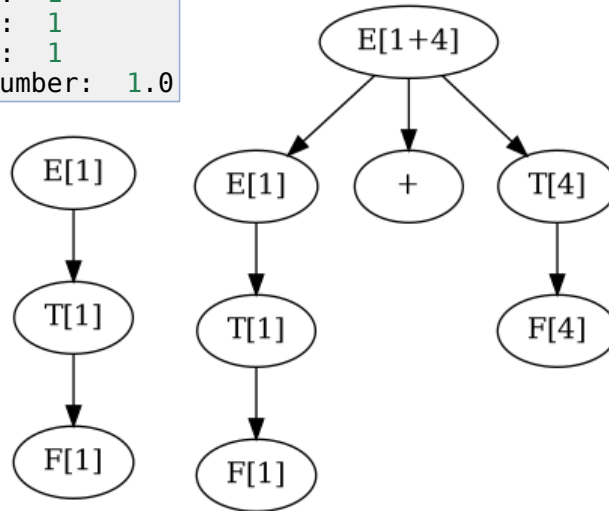
1. $E \rightarrow E + T$
2. $E \rightarrow E - T$
3. $E \rightarrow T$
4. $T \rightarrow T$
5. $T \rightarrow T * F$
6. $T \rightarrow T / F$
7. $T \rightarrow F$
8. $F \rightarrow a$
9. $F \rightarrow (E)$

```

> 1
E: 1
T: 1
F: 1
Number: 1.0
    
```

```

> 1
E: 1
T: 1
F: 1
Number: 1.0
+-: +4*(3-1)
E: 4*(3-1)
T: 4*(3-1)
F: 4*(3-1)
Number: 4.0
    
```

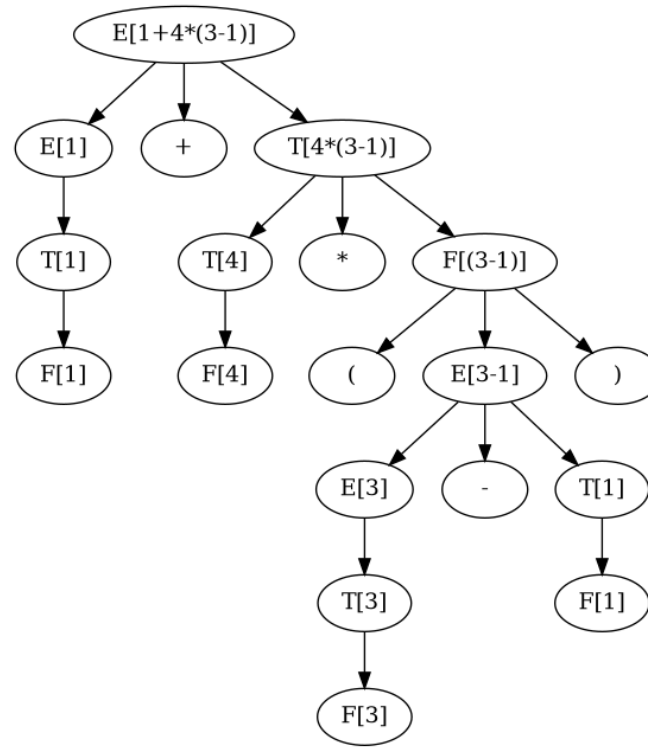


1. $E \rightarrow E + T$
2. $E \rightarrow E - T$
3. $E \rightarrow T$
4. $T \rightarrow T$
5. $T \rightarrow T * F$
6. $T \rightarrow T / F$
7. $T \rightarrow F$
8. $F \rightarrow a$
9. $F \rightarrow (E)$

```

> 1+4*(3-1)
E: 1+4*(3-1)
T: 1+4*(3-1)
F: 1+4*(3-1)
Number: 1.0
+-: +4*(3-1)
E: 4*(3-1)
T: 4*(3-1)
F: 4*(3-1)
Number: 4.0
*/: *(3-1)
T: (3-1)
F: (3-1)
E: 3-1)
T: 3-1)
F: 3-1)
Number: 3.0
+-: -1)
E: 1)
T: 1)
F: 1)
Number: 1.0
True

```



1. $E \rightarrow E + T$
2. $T[1] + T$
3. $F[1] + T$
4. $a[1] + T$
5. $a[1] + T * F$
6. $a[1] + F * F$
7. $a[1] + a[4] * F$
8. $a[1] + a[4] * (E)$
9. $a[1] + a[4] * (E-T)$
10. $a[1] + a[4] * (T-T)$
11. $a[1] + a[4] * (F-T)$
12. $a[1] + a[4] * (a[3]-T)$
13. $a[1] + a[4] * (a[3]-F)$
14. $a[1] + a[4] * (a[3]-a[1])$

Calculator

```
2 def E(s1):
3     [s1, x]=T(s1)
4     while len(s1) > 0 and \
5         (s1[0] == '+' or s1[0] == '-'):
6         if s1[0]=='+' :
7             flag = True
8         else:
9             flag = False;
10        [s1, y] = T(s1[1:])
11        if flag == True:
12            x += y;
13        else:
14            x -= y
15    return [s1, x]
16
17 def T(s1):
18     [s1, x] = F(s1)
19     while len(s1) > 0 and \
20         (s1[0] == '*' or s1[0] == '/'):
21         if s1[0] == '*':
22             flag = True;
23         else:
24             flag = False;
25        [s1, y] = F(s1[1:])
26        if flag == True:
27            x *= y;
28        else:
29            x /= y
30    return [s1, x]
```

```
31 def F(s1):
32     if s1[0] == '(':
33         s1 = s1[1:]
34         [s1, x] = E(s1)
35         if len(s1) == 0 or s1[0] != ')':
36             print('Error: ) missing')
37             raise
38         s1 = s1[1:]
39         return [s1, x]
40     else:
41         a = re.match('[0-9]+(\\.[0-9]+)?', s1)
42         s1 = s1[a.end():]
43         return [s1, float(a.group())]
44
45 st = input('Enter >> ') #st='12+35-2'
46 while st != "":
47     try:
48         [s1, val] = E(st)
49         print(val)
50     except:
51         print('Error')
52     st = input('Enter >> ') #st='12+35-2'
```


Calculator(IV)

```
1 import re, traceback as tb
2 class Lexical:
3
4     def getToken(self):
5         i=self.index
6         res = ['$ ', i];
7         if len(self.s) <= i:
8             return res
9         self.pi = self.index
10        c=self.s[self.index]
11        if c == '*': res[0]='*'
12        elif c == '/': res[0]='/'
13        elif c == '+': res[0]='+'
14        elif c == '-': res[0]='-'
15        elif c == '(': res[0]='('
16        elif c == ')': res[0]=')'
17        elif '0' <= c <= '9':
18            p=r'[0-9]+(\\.[0-9]+)?'
19            a=re.match(p,self.s[i:])
20            x = float(a.group())
21            res=['n', x]
22            self.index += a.end()-1
23        else:
24            raise Exception('UN: '+c)
25        self.index += 1
26        self.ct = res
27        return res
28
29    def __init__(self, s = '4'):
```

```
30        self.ct=['$', 0]; self.s=s
31        self.index=0; self.pi = 0
32
33    def unget(self):
34        self.index = self.pi
35
36    def F(lex):
37        x = 0
38        if lex.ct[0] == 'n':
39            x = lex.ct[1];
40        elif lex.ct[0] == '(':
41            lex.getToken()
42            x = E(lex);
43            print(lex.ct)
44            if lex.ct[0] != ')':
45                raise Exception('')
46        else:
47            raise Exception('F()', lex.ct[0])
48        lex.getToken()
49        return x;
50
51    def T(lex):
52        x = F(lex);
53        p = lex.ct[0]
54        while p in ['*', '/']:
55            lex.getToken()
56            y = F(lex);
57            if p == '*': x *= y
58            else: x /= y;
59        p = lex.ct[0]
```

```
61    return x;
62
63    def E(lex):
64        x = T(lex);
65        p = lex.ct[0]
66        while p in ['+', '-']:
67            lex.getToken()
68            y = T(lex);
69            if p == '+': x += y
70            else: x -= y;
71            p = lex.ct[0]
72        return x;
73
74    def getInput():
75        s = input('Enter>> ')
76        return s.strip()
77
78    s = getInput()
79    while s != '':
80        try:
81            lex = Lexical(s)
82            lex.getToken();
83            print(E(lex));
84        except Exception as e:
85            d=tb.format_exception(e)
86            print("".join(d))
87        s=getInput()
```

Lexial of Calc80 in Python

```
1 import re, traceback as tb
2
3 class RegexMatch:
4
5     def __init__(self, pattern):
6         self.pattern = re.compile(pattern)
7         self.match = None
8
9     def __eq__(self, other):
10        self.match = self.pattern.match(str(other))
11        return self.match is not None
12
13 class Patterns:
14     NUM = RegexMatch(r"[0-9]")
15
16 class yo_error(BaseException):
17     def __init__(self, s, s2=''): self.s=s+str(s2)
18
19 class Lexical:
20     def __init__(self, s = '4'):
21         self.reset(s)
22
23     def reset(self, s):
24         self.ct = ['$ ', 0]; self.s = s
25         self.index = 0; self.pi = 0
26
27     def __str__(self):
28         return f'i:{self.index}, '+\
29             f'ct:{self.ct}, s:{self.s}'
```

```
30 def getToken(self):
31     res = ['$ ', 0]
32     while self.index < len(self.s) \
33         and self.s[self.index] in [' ', '\t']:
34         self.index += 1
35     if len(self.s) <= self.index:
36         return res
37     self.pi = self.index;
38     match self.s[self.index]:
39         case '+': res[0] = '+'
40         case '-': res[0] = '-'
41         case '*': res[0] = '*'
42         case '/': res[0] = '/'
43         case '(': res[0] = '('
44         case ')': res[0] = ')'
45     case Patterns.NUM:
46         p = r'[0-9]+(\.[0-9]+)?'
47         a = re.match(p,
48             self.s[self.index:])
49         res = ['n', float(a.group())]
50         self.index += a.end() - 1;
51     case _:
52         raise yo_error('lexical:', self.s)
53     self.index += 1
54     self.ct = res
55     return res
```

Parser of Calc80 in Python

```
57 class Parser:
58     def __init__(self, lex):
59         self.lex = lex
60
61     def F(self):
62         x = 0
63         match self.lex.ct[0]:
64             case 'n': x=self.lex.ct[1]
65             case '(':
66                 self.lex.getToken()
67                 x = self.E();
68                 if self.lex.ct[0] != ')':
69                     raise yo_error('):: ',self.lex)
70             case _:
71                 raise yo_error('F()',
72                               self.lex)
73         self.lex.getToken()
74         return x;
75
76     def E(self):
77         x = self.T();
78         while self.lex.ct[0] in ['+', '-']:
79             p = self.lex.ct[0]
80             self.lex.getToken()
81             y = self.T();
82             if p == '+': x += y
```

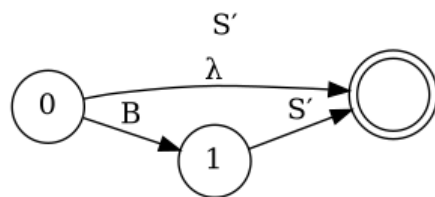
```
84         else: x -= y;
85         return x;
86
87     def T(self):
88         x = self.F();
89         while self.lex.ct[0] in ['*', '/']:
90             p=self.lex.ct[0]
91             self.lex.getToken()
92             y = self.F();
93             if p == '*': x *= y
94             else:      x /= y;
95         return x;
96
97     def getInput():
98         s = input('Enter>> ')
99         return s.strip()
100
101 if __name__ == '__main__':
102     lex = Lexical()
103     parse=Parser(lex)
104     while True:
105         s = getInput(); lex.reset(s)
106         if len(s.strip()) == 0 :break
107         try:
108             lex.getToken();
109             print(parse.E());
110         except yo_error as yo:
111             print('yo_error ', yo.s);
112         except Exception as e:
113             dt=tb.format_exception(e)
114             print("".join(td))
```

Another Grammar(I)

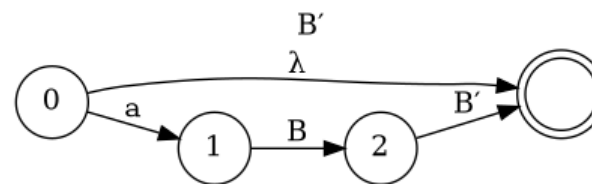
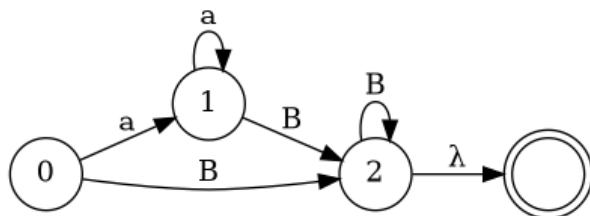
1. $S \rightarrow AB \mid SB \mid B$
2. $A \rightarrow aA$
3. $B \rightarrow bB \mid BaB \mid ab$

Eliminate Left Recursion

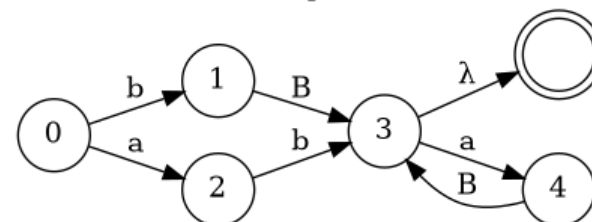
1. $S \rightarrow AB \mid SB \mid B$
 - $S \rightarrow ABS' \mid BS'$
 - $S' \rightarrow BS' \mid \lambda$
2. $B \rightarrow bB \mid BaB \mid ab$
 - $B \rightarrow bBB' \mid abB'$
 - $B' \rightarrow aBB' \mid \lambda$



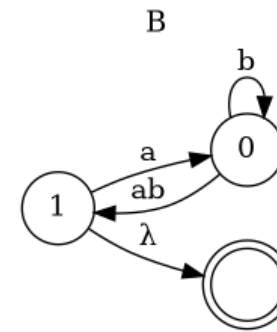
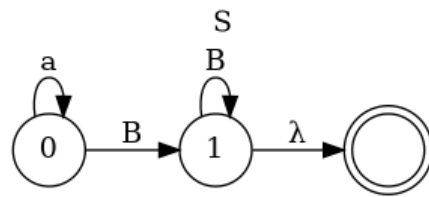
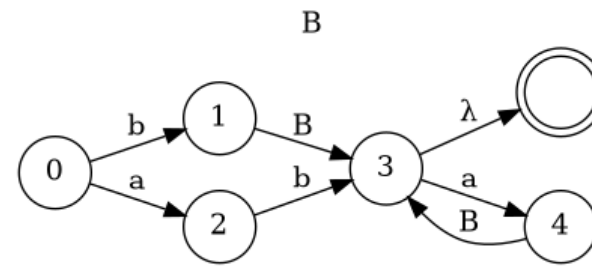
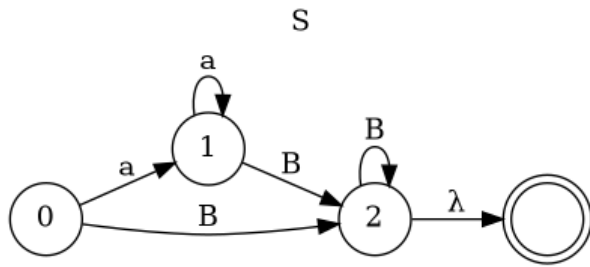
S(S + Simplified S' + Simplified A)

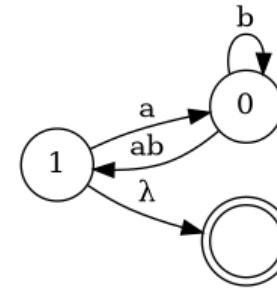
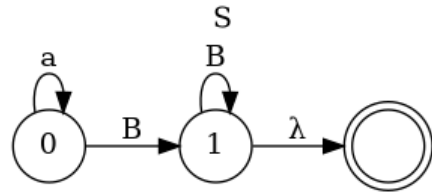


B + Simplified B'



Another Grammar(II)





```

1 class Parser:
2     def __init__(self, text):
3         self.text = text; self.pos = 0
4     def cur_ch(self):
5         if self.pos < len(self.text):
6             return self.text[self.pos]
7         return None
8     def match(self, token):
9         if self.text.startswith(token, self.pos):
10            self.pos += len(token); return True
11        return False
12    def error(self, message):
13        raise SyntaxError(
14            f"{message} at position {self.pos}")
15    def S(self):
16        while self.match('a'): print("S: consume 'a'")
17        self.B()
18        while self.cur_ch() in ('a', 'b'):
19            old_pos = self.pos

```

```

20            try: self.B()
21            except SyntaxError: self.pos = old_pos; break
22    def B(self):
23        while self.match('b'): print("B: consume 'b'")
24        if self.match('ab'): print("B: consume 'ab'")
25        else: self.error("Expected 'ab'")
26        while self.match('a'): self.B()
27    def parse(self):
28        self.S()
29        if self.pos != len(self.text):
30            self.error("Unexpected input")
31        print("\nInput accepted.")
32
33 samples = ["ab", "aab", "bbab", "abbab", "aaabbab",]
34 for s in samples:
35     print("\n=====Input:", s)
36     try: parser = Parser(s); parser.parse()
37     except SyntaxError as e: print("Syntax Error:", e)

```

1. $S \rightarrow AB \mid SB \mid B \{ S \rightarrow ABS' \mid BS', S' \rightarrow BS' \mid \lambda \}$
2. $B \rightarrow bB \mid BaB \mid ab \{ B \rightarrow bBB' \mid abB', B' \rightarrow aBB' \mid \lambda \}$

- $\text{first}(S) = \{a\} \cup \text{first}(B)$
- $\text{first}(B) = \{a, \text{first}(ab)\} = \{a, b\}$
- $\text{first}(S) = \{a, b\}$

Lexical C++

```
1 #include<iostream>
2 #include<cstdlib>
3
4 enum TokenType{
5     NUMBER, PLUS, MINUS, MUL, DIV,
6     LPAR, RPAR, END_OF_INPUT
7 };
8 struct Token
9 {TokenType type; int n; double r;};
10
11 void error(const std::string msg){
12     std::cout << msg << std::endl;
13     exit(0);
14 }
15
16 Token t;
17 char buffer[200];
18 int tindex = 0;
19
20 void E(void);
21 void T(void);
22 void F(void);
23
24 void read(void){
25     std::cout << "Enter expression"
26     << std::endl;
27     std::cin>>buffer;
28 }
29
30 Token getToken(void){
31     static int number_of_call = 0;
```

```
31     number_of_call ++;
32     if(buffer[tindex] == '(')
33         t.type = LPAR;
34     else if(buffer[tindex] == ')')
35         t.type = RPAR;
36     else if(buffer[tindex] == '+')
37         t.type = PLUS;
38     else if(buffer[tindex] == '-')
39         t.type = MINUS;
40     else if(buffer[tindex] == '*')
41         t.type = MUL;
42     else if(buffer[tindex] == '/')
43         t.type = DIV;
44     else if(buffer[tindex] >= '0' &&
45             buffer[tindex]<='9'){
46         t.type=NUMBER;
47         t.n = 0;
48         for(; buffer[tindex] >= '0'
49             && buffer[tindex] <='9'; tindex++)
50             t.n = t.n * 10 + buffer[tindex] - 48;
51         tindex --;
52     }
53     else if(buffer[tindex] == '\\0')
54         t.type = END_OF_INPUT;
55     else
56         error("unknown character");
57     //std::cout << t.type<<'\t'<<tindex<<'\t'
58     //<<number_of_call<<'\t'<<t.n<<std::endl;
59     tindex++;
60     return t;
61 }
```


Calculator C++

```
63 void A(void){
64     t = getToken();
65     E();
66     if(t.type != END_OF_INPUT)
67         error("unrecognized token");
68     else
69         std::cout << "Accept" << std::endl;
70 }
71
72 void E(void){
73     T();
74     while(t.type == PLUS || t.type == MINUS){
75         t = getToken();
76         T();
77     }
78 }
79
80 void T(void){
81     F();
82     while(t.type == MUL || t.type == DIV ){
83         t = getToken();
84         F();
85     }
86 }
87
88 void F(void){
89     if(t.type == LPAR){
```

```
90         t = getToken();
91         E();
92         if(t.type != RPAR)
93             error(" ) is missing");
94     }
95     else if(t.type==NUMBER)
96         ;
97     else
98         error("Wrong token ");
99     t = getToken();
100 }
101
102 void tryGetToken(void){
103     do{
104         t = getToken();
105     }while(t.type != END_OF_INPUT);}
106
107 int main(){
108     read(); /*tryGetToken();*/
109     A();
110     return 0;
111 }/*2-(3-2)/(3-(2-1)/(5-2*2))-1+2*/
```

Simple Programming Language

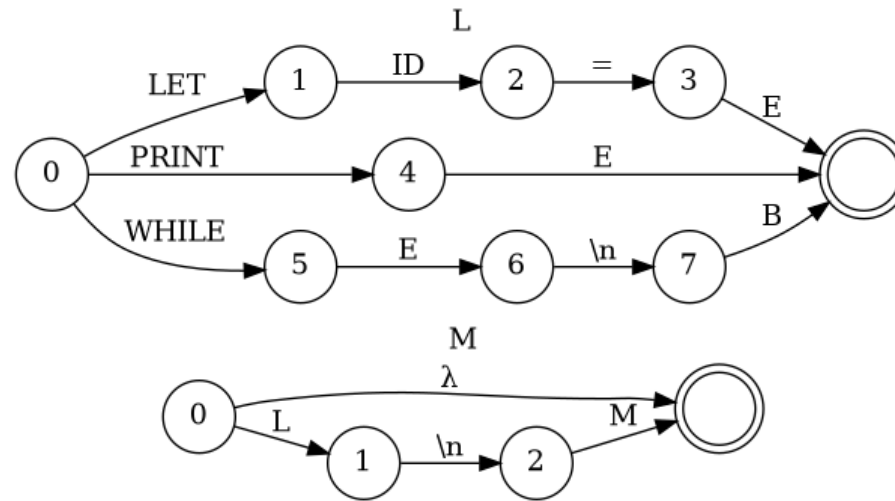
```
1 BEGIN
2   LET I = 12
3   PRINT I
4   LET I = I - (2 - (3-2)/(3 - (2-1)/(5-2*2)))-1+2)
5   PRINT I+ 2 - (3-2)/(3 - (2-1)/(5-2*2)) -1+2
6   LET I=19
7   WHILE I-8
8     BEGIN
9       PRINT I
10      LET I=I-1
11    END
12    LET I=3
13    WHILE I
14      BEGIN
15        PRINT I
16        LET J=6
17        WHILE J
18          BEGIN
19            LET J=J-2
20            PRINT J
21          END
22          LET I=I-1
23        END
24    END
```

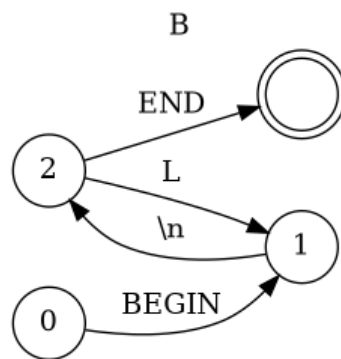
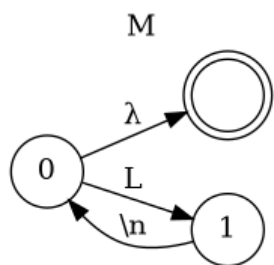
1. $A \rightarrow B \text{ 'EOF'}$
2. $B \rightarrow \text{'BEGIN' '}\backslash\text{n' M 'END'}$
3. $M \rightarrow L \text{ '}\backslash\text{n' M} \mid \lambda$
4. $L \rightarrow \text{'LET' ID} = E$
5. $L \rightarrow \text{'PRINT' E}$
6. $L \rightarrow \text{'WHILE' E '}\backslash\text{n' B}$
7. $E \rightarrow T E'$
8. $E' \rightarrow + T E' \mid - T E' \mid \lambda$
9. $T \rightarrow F T'$
10. $T' \rightarrow * F T' \mid / F T' \mid \lambda$
11. $F \rightarrow \text{'NUMBER'}$
12. $F \rightarrow \text{'ID'}$
13. $F \rightarrow (E)$

src/rd/simple_language_1/cpp/simple_language_1.cpp

Chart for simple programming Language

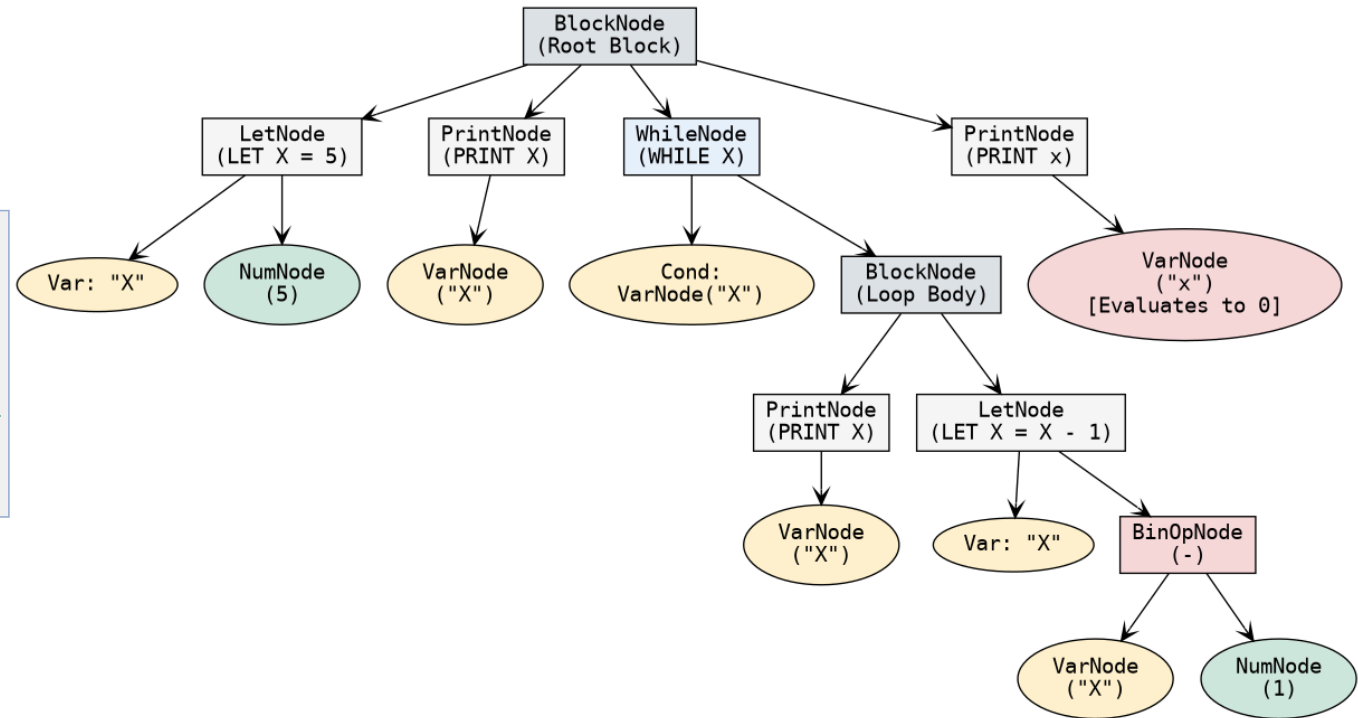
1. $A \rightarrow B \text{'EOF'}$
2. $B \rightarrow \text{'BEGIN' } \backslash n \text{' M 'END'}$
3. $M \rightarrow L \backslash n \text{' M } | \lambda$
4. $L \rightarrow \text{'LET' ID' = E}$
5. $L \rightarrow \text{'PRINT' E}$
6. $L \rightarrow \text{'WHILE' E } \backslash n \text{' B}$
7. $E \rightarrow T \text{' E'}$
8. $E' \rightarrow + T \text{' E' } | - T \text{' E' } | \lambda$
9. $T \rightarrow F \text{' T'}$
10. $T' \rightarrow * F \text{' T' } | / F \text{' T' } | \lambda$
11. $F \rightarrow \text{'NUMBER'}$
12. $F \rightarrow \text{'ID'}$
13. $F \rightarrow (\text{' E' })$





Creating Syntax Tree (Top Down)

```
BEGIN
  LET X = 5
  PRINT X
  WHILE X
  BEGIN
    PRINT X
    LET X = X - 1
  END
  PRINT x
END
```



```

1 from sly import Lexer
2 class BasicLexer(Lexer):
3     tokens = {
4         BEGIN, END, LET, PRINT, WHILE, ID, NUMBER, NEWLINE,
5     }
6     literals = {"+", "-", "*", "/", "=", "(", ")"}
7     ignore = " \t"
8     @_("["A-Za-z_][A-Za-z0-9_]*")
9     def ID(self, t):
10         v = {"BEGIN", "END", "LET", "PRINT", "WHILE"}
11         if t.value in v: t.type = t.value
12         return t
13     @_("\d+")
14     def NUMBER(self, t):
15         t.value = int(t.value); return t
16     @_("\n+")
17     def NEWLINE(self, t):
18         self.lineno += len(t.value); return t
19     def error(self, t): raise SyntaxError("Err")
20 class BlockNode:
21     def __init__(self, s): self.s = s
22     def eval(self, env):
23         for st in self.s: st.eval(env)
24 class LetNode:
25     def __init__(self, n, e): self.n, self.e = n, e
26     def eval(self, env): env[self.n] = self.e.eval(env)
27 class PrintNode:
28     def __init__(self, e): self.e = e
29     def eval(self, env): print(self.e.eval(env))
30 class WhileNode:
31     def __init__(self, c, b): self.c, self.b = c, b

```

```

31     def eval(self, env):
32         while self.c.eval(env) != 0: self.b.eval(env)
33 class BinOpNode:
34     def __init__(self, o, l, r):
35         self.o, self.l, self.r = o, l, r
36     def eval(self, env):
37         l, r = self.l.eval(env), self.r.eval(env)
38         if self.o == "+": return l + r
39         if self.o == "-": return l - r
40         if self.o == "*": return l * r
41         if self.o == "/": return l // r
42 class NumNode:
43     def __init__(self, v): self.v = v
44     def eval(self, env): return self.v
45 class VarNode:
46     def __init__(self, n): self.n = n
47     def eval(self, env): return env.get(self.n, 0)
48 class Interpreter:
49     def __init__(self, src):
50         self.tk = list(BasicLexer().tokenize(src))
51         self.p = 0; self.vars = {}
52     def cur(self):
53         if self.p < len(self.tk): return self.tk[self.p]
54         return None
55     def cur_t(self):
56         c = self.cur()
57         return c.type if c else "EOF"
58     def advance(self):
59         t = self.cur(); self.p += 1; return t
60     def match(self, k):
61         if self.cur_t() == k: return self.advance()
62         return None

```

```

63 def expect(self, k):
64     t = self.match(k)
65     if not t: raise SyntaxError("Err")
66     return t
67 def parse(self):
68     while self.cur_t() == "NEWLINE": self.advance()
69     ast = self.B() # Annotated Syntax Tree
70     while self.cur_t() == "NEWLINE": self.advance()
71     return ast
72 def B(self):
73     self.expect("BEGIN"); self.expect("NEWLINE")
74     st = self.M(); self.expect("END")
75     return BlockNode(st)
76 def M(self):
77     st = []; v = ("LET", "PRINT", "WHILE", "NEWLINE")
78     while self.cur_t() in v:
79         if self.cur_t() == "NEWLINE":
80             self.advance()
81             continue
82         st.append(self.L())
83         if self.cur_t() == "NEWLINE": self.advance()
84     return st
85 def L(self):
86     if self.match("LET"):
87         n = self.expect("ID").value
88         self.expect("=")
89         return LetNode(n, self.E())
90     if self.match("PRINT"):
91         return PrintNode(self.E())
92     if self.match("WHILE"):
93         c = self.E()
94         while self.cur_t() == "NEWLINE": self.advance()

```

```

96         return WhileNode(c, self.B())
97     raise SyntaxError("Err")
98 def E(self):
99     n = self.T()
100     while self.cur_t() in ("+", "-"):
101         o = self.advance().type
102         n = BinOpNode(o, n, self.T())
103     return n
104 def T(self):
105     n = self.F()
106     while self.cur_t() in ("*", "/"):
107         o = self.advance().type
108         n = BinOpNode(o, n, self.F())
109     return n
110 def F(self):
111     t = self.cur()
112     if not t: raise SyntaxError("Err")
113     if t.type == "NUMBER":
114         self.advance()
115         return NumNode(t.value)
116     if t.type == "ID":
117         self.advance()
118         return VarNode(t.value)
119     if t.type == "(":
120         self.advance(); n = self.E(); self.expect
121         return n
122     raise SyntaxError("Err")
123 source = """BEGIN
124 LET X = 5
125 PRINT X
126
127 WHILE X

```

List of Related Files

- src/rd/simple_language_1/
 1. simple_language_1.py
 - created by chatgpt.com
 2. simple_language20.py
 - Edited by Grok AI and Gemeni AI
 3. simple_language30.py
 - reformat by Gemini AI
 4. simple_language50.py
 - Add argparse added by me
 5. sly folder of sly tool
- src/rd/simple_language_1/cpp/
 1. simple_language20.cpp
 - Gemini Ai created (Flawed)
 2. simple_language30.cpp
 - Gemini Ai reformat (Flawed)
 3. simple_language50.cpp
 - Change by hand to support arguments (Flawed)
 4. simple_language_1.cpp
 - My old code for interpreted
 - Compiles and works fine after decades
 5. simple_language_2.cpp
 - Reformated of my old code by hand
 - I like to update it more but I will not able to.

End

